

Iteration and Loop Control Structures in C++

📌 Definition to Remember

Iteration structures in C++ automate repetitive algorithms through for, while, do-while, and range-based for loops, optimized by compiler branch prediction.

1. Concept Overview & Fundamentals 🧠

Iteration and Loop Control Structures in C++ provides modern high-performance capabilities combining low-level hardware control with high-level abstractions, object-oriented paradigms, and Standard Template Library (STL) utilities.

In modern C++ (C++11 through C++20):

- **Zero-Overhead Principle:** What you don't use, you don't pay for; what you do use, you couldn't hand code any better.
- **Type Safety & Streams:** Type-safe stream I/O (`std::cin` , `std::cout`) prevents runtime format mismatch vulnerabilities.
- **Resource Acquisition Is Initialization (RAII):** Automatic resource management via constructors, destructors, and smart pointers.

2. Technical Architecture & Memory Mechanics ⚙️

C++ Runtime Execution Architecture

```
+-----+
|                STACK SEGMENT                |
| - Local scope objects, references, RAII stack frames |
| - Deterministic automatic destruction upon scope exit |
+-----+
|
| v
| ^
|
+-----+
|                HEAP / FREE STORE              |
| - Dynamic objects allocated via new, std::vector, etc. |
| - Managed via delete / smart pointers          |
+-----+
|                STATIC & GLOBAL MEMORY          |
| - Global objects, static class variables, string literals |
+-----+
|                CODE / TEXT SEGMENT              |
| - Executable machine code, inlined functions, templates |
+-----+
```

Modern C++ Feature Comparison

Modern C++ Paradigm	Legacy Equivalent	Engineering Advantage	Performance Impact
<code>std::cin</code> / <code>std::cout</code>	<code>scanf</code> / <code>printf</code>	Type-safe, extensible for custom classes	Buffered I/O (optimizable via <code>ios::sync_with_stdio(0)</code>)
References (<code>Type&</code>)	Pointers (<code>Type*</code>)	Non-nullable, clean syntax without dereferencing	Direct alias; zero runtime overhead
<code>std::vector<T></code>	Raw Dynamic Array (<code>malloc</code>)	Automatic resizing, bounds safety, memory management	Amortized O(1) push_back, contiguous cache
<code>nullptr</code>	<code>NULL</code> / <code>0</code>	Strongly typed pointer literal preventing integer overload bugs	Zero overhead

3. Core Syntax & Implementation Patterns

Modern C++ Template Structure

```
#include <iostream>
#include <vector>
#include <string>
#include <algorithm>

// Fast I/O and standard namespace configuration
int main() {
    std::ios_base::sync_with_stdio(false);
    std::cin.tie(NULL);

    std::cout << "Executing C++ Module: Iteration & Loop Control Structure" << "\n";

    return 0;
}
```

4. Real-World Applications & System Engineering

- Quantitative Finance & HFT:** High-frequency trading platforms use modern C++ for microsecond-level execution.
- Game Engine Architecture:** Unreal Engine, Unity Core, and AAA game engines utilize C++ for real-time 3D rendering.
- Deep Learning Frameworks:** Core tensor compute backends (PyTorch, TensorFlow, ONNX Runtime) are written in C++ and CUDA.
- Web Browsers & Compilers:** Google Chrome (V8 engine), LLVM/Clang, and GCC are implemented in C++.

5. Comprehensive Solved Problems & Code Walkthroughs

Problem 1: ArmstrongNumber

Problem Description & Objective:

Implementation of ArmstrongNumber

C++ Code Implementation:

```

/*
 * Question:
 * Program to check if a number is an Armstrong number.
 */

/*
 * =====
 * CONCEPT EXPLORATION & ALGORITHM BREAKDOWN
 * =====
 * 1. WHICH CONCEPT IS USED?
 *    - Digit Extraction, Power Calculation (`std::pow`), and Loop Iteration.
 *
 * 2. WHY IT IS USED?
 *    - An Armstrong number of n digits equals the sum of its digits raised to the nth power. Digit
    extraction is essential for numeric decomposition.
 *
 * 3. HOW IT IS USED?
 *    - Uses `while (temp != 0)` to count digits and another `while` loop to extract digits using `% 10`
    and calculate `pow(remainder, n)`.
 *
 * 4. ALGORITHM EXPLANATION:
 * *   Step 1: Input integer `num` and save in `originalNum`.
 *   Step 2: Count number of digits `n` using loop division `temp /= 10`.
 *   Step 3: Reset `temp = num` and calculate sum of digit^n powers.
 *   Step 4: If `sum == originalNum`, print Armstrong; else Not Armstrong.
 * =====
 */

#include <iostream>
#include <cmath>

// Use standard library namespace globally
using namespace std;

// --- Main Execution Entry Point ---
int main() {
    int num, originalNum, remainder, result = 0, n = 0;

    // Output formatted data to console stream
    cout << "Enter an integer: ";
    // Read user input from standard input stream
    cin >> num;

    originalNum = num;

    int temp = num;
    // Loop execution block
    while (temp != 0) {
        temp /= 10;
        ++n;
    }

    temp = num;
    // Loop execution block
    while (temp != 0) {
        remainder = temp % 10;
        result += round(pow(remainder, n));
    }
}

```

```
        temp /= 10;
    }

    // Evaluate conditional decision logic
    if (result == originalNum)
        // Output formatted data to console stream
        cout << originalNum << " is an Armstrong number." << endl;
    // Alternative conditional branch
    else
        // Output formatted data to console stream
        cout << originalNum << " is NOT an Armstrong number." << endl;

    // Return zero code indicating successful program finish
    return 0;
}
```

Problem 2: EvenPrint

Problem Description & Objective:

Implementation of EvenPrint

C++ Code Implementation:

```

/*
 * Question:
 * Program using continue to print only even numbers up to N.
 */

/*
 * =====
 * CONCEPT EXPLORATION & ALGORITHM BREAKDOWN
 * =====
 * 1. WHICH CONCEPT IS USED?
 *    - Loop Control Flow Modification with the `continue` statement.
 *
 * 2. WHY IT IS USED?
 *    - `continue` skips the current loop iteration immediately when an odd number is encountered,
    bypassing the print step.
 *
 * 3. HOW IT IS USED?
 *    - `if (i % 2 != 0) continue;` skips odd numbers in a `for` loop.
 *
 * 4. ALGORITHM EXPLANATION:
 * *   Step 1: Input limit `n`.
 *   Step 2: Loop `i` from 1 to `n`.
 *   Step 3: If `i` is odd (`i % 2 != 0`), execute `continue`.
 *   Step 4: Otherwise, print `i`.
 * =====
 */

#include <iostream>

// Use standard library namespace globally
using namespace std;

// --- Main Execution Entry Point ---
int main() {
    int n;
    // Output formatted data to console stream
    cout << "Enter limit N: ";
    // Read user input from standard input stream
    cin >> n;

    // Output formatted data to console stream
    cout << "Even numbers from 1 to " << n << ":" << endl;
    // Loop execution block
    for (int i = 1; i <= n; i++) {
        // Evaluate conditional decision logic
        if (i % 2 != 0) {
            continue;
        }
        // Output formatted data to console stream
        cout << i << " ";
    }
    // Output formatted data to console stream
    cout << endl;

    // Return zero code indicating successful program finish

```

```
    return 0;  
}
```

Problem 3: Factorial

Problem Description & Objective:

Implementation of Factorial

C++ Code Implementation:

```

/*
 * Question:
 * Write a function that calculates the factorial of a given number.
 */

/*
 * =====
 * CONCEPT EXPLORATION & ALGORITHM BREAKDOWN
 * =====
 * 1. WHICH CONCEPT IS USED?
 *    - Iterative Accumulation Function & 64-bit Integer Types (`long long`).
 *
 * 2. WHY IT IS USED?
 *    - `long long` prevents integer overflow because factorial values grow extremely fast.
 *
 * 3. HOW IT IS USED?
 *    - `for (int i = 1; i <= n; i++) fact *= i;` accumulates the product.
 *
 * 4. ALGORITHM EXPLANATION:
 * *   Step 1: Input non-negative integer `num`.
 * *   Step 2: Call `calculateFactorial(num)`.
 * *   Step 3: Inside function, multiply running product `fact` by `1..n`.
 * *   Step 4: Return and display factorial value.
 * =====
 */

#include <iostream>

// Use standard library namespace globally
using namespace std;

long long calculateFactorial(int n) {
    // Evaluate conditional decision logic
    if (n < 0) return -1;
    long long fact = 1;
    // Loop execution block
    for (int i = 1; i <= n; i++) {
        fact *= i;
    }
    // Return computed result from function
    return fact;
}

// --- Main Execution Entry Point ---
int main() {
    int num;
    // Output formatted data to console stream
    cout << "Enter a non-negative integer: ";
    // Read user input from standard input stream
    cin >> num;

    long long result = calculateFactorial(num);
    // Evaluate conditional decision logic
    if (result == -1) {
        // Output formatted data to console stream
        cout << "Factorial is not defined for negative numbers." << endl;
        // Alternative conditional branch
    }
}

```

```
} else {  
    // Output formatted data to console stream  
    cout << "Factorial of " << num << " = " << result << endl;  
}  
  
// Return zero code indicating successful program finish  
return 0;  
}
```

Problem 4: Fibonacci

Problem Description & Objective:

Implementation of Fibonacci

C++ Code Implementation:

```

/*
 * Question:
 * Program to print Fibonacci series up to a certain number of terms.
 */

/*
 * =====
 * CONCEPT EXPLORATION & ALGORITHM BREAKDOWN
 * =====
 * 1. WHICH CONCEPT IS USED?
 *    - Iterative State Swapping (Variable Update Sequence).
 *
 * 2. WHY IT IS USED?
 *    - Computes each Fibonacci number sequentially in O(N) linear time and O(1) space.
 *
 * 3. HOW IT IS USED?
 *    - `nextTerm = t1 + t2; t1 = t2; t2 = nextTerm;` updates consecutive terms.
 *
 * 4. ALGORITHM EXPLANATION:
 * *   Step 1: Input term count `n`.
 * *   Step 2: Initialize `t1 = 0` and `t2 = 1`.
 * *   Step 3: Loop `i` from 1 to `n`: print term, compute `nextTerm = t1 + t2`, shift `t1 = t2`, `t2 =
nextTerm`.
 *   Step 4: Print series.
 * =====
 */

#include <iostream>

// Use standard library namespace globally
using namespace std;

// --- Main Execution Entry Point ---
int main() {
    int n;
    // Output formatted data to console stream
    cout << "Enter the number of terms: ";
    // Read user input from standard input stream
    cin >> n;

    long long t1 = 0, t2 = 1, nextTerm = 0;

    // Output formatted data to console stream
    cout << "Fibonacci Series: ";
    // Loop execution block
    for (int i = 1; i <= n; ++i) {
        // Evaluate conditional decision logic
        if (i == 1) {
            // Output formatted data to console stream
            cout << t1 << " ";
            continue;
        }
        // Evaluate conditional decision logic
        if (i == 2) {
            // Output formatted data to console stream
            cout << t2 << " ";
            continue;
        }
    }
}

```

```
    }
    nextTerm = t1 + t2;
    t1 = t2;
    t2 = nextTerm;
    // Output formatted data to console stream
    cout << nextTerm << " ";
}
// Output formatted data to console stream
cout << endl;

// Return zero code indicating successful program finish
return 0;
}
```

Problem 5: GCD

Problem Description & Objective:

Implementation of GCD

C++ Code Implementation:

```

/*
 * Question:
 * Create a program to find the Greatest Common Divisor (GCD) of two integers.
 */

/*
 * =====
 * CONCEPT EXPLORATION & ALGORITHM BREAKDOWN
 * =====
 * 1. WHICH CONCEPT IS USED?
 *    - Euclidean Algorithm with Modulo Iteration.
 *
 * 2. WHY IT IS USED?
 *    - Reduces GCD time complexity to logarithmic  $O(\log(\min(A,B)))$  time compared to slow  $O(N)$  trial
    division.
 *
 * 3. HOW IT IS USED?
 *    - `while (b != 0) { temp = b; b = a % b; a = temp; }`
 *
 * 4. ALGORITHM EXPLANATION:
 * *   Step 1: Input integers `num1` and `num2`.
 *   Step 2: Pass numbers to `findGCD(a, b)`.
 *   Step 3: In loop `while (b != 0)`: calculate remainder `a % b`, update `a = b`, `b = remainder`.
 *   Step 4: Return `a` as GCD.
 * =====
 */

#include <iostream>

// Use standard library namespace globally
using namespace std;

int findGCD(int a, int b) {
    // Loop execution block
    while (b != 0) {
        int temp = b;
        b = a % b;
        a = temp;
    }
    // Return computed result from function
    return a;
}

// --- Main Execution Entry Point ---
int main() {
    int num1, num2;
    // Output formatted data to console stream
    cout << "Enter two integers: ";
    // Read user input from standard input stream
    cin >> num1 >> num2;

    // Output formatted data to console stream
    cout << "GCD of " << num1 << " and " << num2 << " is: " << findGCD(num1, num2) << endl;

    // Return zero code indicating successful program finish

```

```
    return 0;  
}
```

Problem 6: InfiniteLoop

Problem Description & Objective:

Implementation of InfiniteLoop

C++ Code Implementation:

```

/*
 * Question:
 * Program using infinite loop and break statement.
 */

/*
 * =====
 * CONCEPT EXPLORATION & ALGORITHM BREAKDOWN
 * =====
 * 1. WHICH CONCEPT IS USED?
 *    - Sentinel-Controlled Loop Termination (`while(true)` + `break`).
 *
 * 2. WHY IT IS USED?
 *    - Useful when loop exit conditions depend on dynamic runtime input rather than a fixed iteration
count.
 *
 * 3. HOW IT IS USED?
 *    - `while(true)` runs infinitely until `if (input == 0) break;` triggers.
 *
 * 4. ALGORITHM EXPLANATION:
 *    * Step 1: Enter `while(true)` loop.
 *    * Step 2: Read user integer input.
 *    * Step 3: If input is 0, execute `break` to exit loop immediately.
 *    * Step 4: Otherwise, print input and continue.
 * =====
 */

#include <iostream>

// Use standard library namespace globally
using namespace std;

// --- Main Execution Entry Point ---
int main() {
    int number;
    // Output formatted data to console stream
    cout << "Entering infinite loop. Type '0' to stop." << endl;

    // Loop execution block
    while (true) {
        // Output formatted data to console stream
        cout << "Enter a number: ";
        // Read user input from standard input stream
        cin >> number;

        // Evaluate conditional decision logic
        if (number == 0) {
            // Output formatted data to console stream
            cout << "Break statement executed. Exiting loop." << endl;
            break;
        }

        // Output formatted data to console stream
        cout << "You entered: " << number << endl;
    }

    // Return zero code indicating successful program finish

```

```
    return 0;  
}
```

Problem 7: LCM

Problem Description & Objective:

Implementation of LCM

C++ Code Implementation:

```

/*
 * Question:
 * Create a program to find the Least Common Multiple (LCM) of two numbers.
 */

/*
 * =====
 * CONCEPT EXPLORATION & ALGORITHM BREAKDOWN
 * =====
 * 1. WHICH CONCEPT IS USED?
 *    - GCD-based LCM Mathematical Formula.
 *
 * 2. WHY IT IS USED?
 *    - Calculates LCM efficiently via `LCM(a, b) = (a * b) / GCD(a, b)`.
 *
 * 3. HOW IT IS USED?
 *    - Computes GCD first recursively, then divides `(a / gcd) * b` to prevent arithmetic overflow.
 *
 * 4. ALGORITHM EXPLANATION:
 *    * Step 1: Input positive integers `n1` and `n2`.
 *    * Step 2: Compute GCD using recursive function `gcd(a, b)`.
 *    * Step 3: Calculate `lcm = (n1 / gcd(n1, n2)) * n2`.
 *    * Step 4: Display `lcm`.
 * =====
 */

#include <iostream>

// Use standard library namespace globally
using namespace std;

int gcd(int a, int b) {
    // Return computed result from function
    return (b == 0) ? a : gcd(b, a % b);
}

// --- Main Execution Entry Point ---
int main() {
    int n1, n2;
    // Output formatted data to console stream
    cout << "Enter two positive integers: ";
    // Read user input from standard input stream
    cin >> n1 >> n2;

    int lcm = (n1 / gcd(n1, n2)) * n2;

    // Output formatted data to console stream
    cout << "LCM of " << n1 << " and " << n2 << " is: " << lcm << endl;

    // Return zero code indicating successful program finish
    return 0;
}

```

Problem 8: Multiplication

Problem Description & Objective:

Implementation of Multiplication

C++ Code Implementation:

```

/*
 * Question:
 * Develop a program that prints the multiplication table for a given number.
 */

/*
 * =====
 * CONCEPT EXPLORATION & ALGORITHM BREAKDOWN
 * =====
 * 1. WHICH CONCEPT IS USED?
 *    - Count-Controlled `for` Loop.
 *
 * 2. WHY IT IS USED?
 *    - Iterates over a fixed range 1 to 10 to display multiplication facts.
 *
 * 3. HOW IT IS USED?
 *    - `for (int i = 1; i <= 10; ++i)` calculates `num * i`.
 *
 * 4. ALGORITHM EXPLANATION:
 *    * Step 1: Input integer `num`.
 *    * Step 2: Loop `i` from 1 to 10.
 *    * Step 3: Print `num * i`.
 *    * Step 4: End loop.
 * =====
 */

#include <iostream>

// Use standard library namespace globally
using namespace std;

// --- Main Execution Entry Point ---
int main() {
    int num;
    // Output formatted data to console stream
    cout << "Enter an integer: ";
    // Read user input from standard input stream
    cin >> num;

    // Output formatted data to console stream
    cout << "Multiplication Table for " << num << ":\n" << endl;
    // Loop execution block
    for (int i = 1; i <= 10; ++i) {
        // Output formatted data to console stream
        cout << num << " * " << i << " = " << (num * i) << endl;
    }

    // Return zero code indicating successful program finish
    return 0;
}

```

Problem 9: OddSum

Problem Description & Objective:

Implementation of OddSum

C++ Code Implementation:

```
/*
 * Question:
 * Create a program to sum all odd numbers from 1 to a specified number N.
 */

/*
 * =====
 * CONCEPT EXPLORATION & ALGORITHM BREAKDOWN
 * =====
 * 1. WHICH CONCEPT IS USED?
 *    - Loop Stepping (`i += 2`).
 *
 * 2. WHY IT IS USED?
 *    - Incrementing by 2 skips even numbers automatically, doubling efficiency over checking `i % 2 !=
0`.
 *
 * 3. HOW IT IS USED?
 *    - `for (int i = 1; i <= n; i += 2) sum += i;`
 *
 * 4. ALGORITHM EXPLANATION:
 * *   Step 1: Read limit `n`.
 *   Step 2: Loop `i` starting at 1, incrementing by 2 up to `n`.
 *   Step 3: Add `i` to accumulator `sum`.
 *   Step 4: Print total `sum`.
 * =====
 */

#include <iostream>

// Use standard library namespace globally
using namespace std;

// --- Main Execution Entry Point ---
int main() {
    int n, sum = 0;
    // Output formatted data to console stream
    cout << "Enter upper limit N: ";
    // Read user input from standard input stream
    cin >> n;

    // Loop execution block
    for (int i = 1; i <= n; i += 2) {
        sum += i;
    }

    // Output formatted data to console stream
    cout << "Sum of all odd numbers from 1 to " << n << " is: " << sum << endl;

    // Return zero code indicating successful program finish
    return 0;
}
```

Problem 10: PalindromeNumber

Problem Description & Objective:

Implementation of PalindromeNumber

C++ Code Implementation:

```

/*
 * Question:
 * Program to check if a number is palindrome.
 */

/*
 * =====
 * CONCEPT EXPLORATION & ALGORITHM BREAKDOWN
 * =====
 * 1. WHICH CONCEPT IS USED?
 *    - Number Reversal Algorithm.
 *
 * 2. WHY IT IS USED?
 *    - Reversing digits allows direct comparison between original and inverted value.
 *
 * 3. HOW IT IS USED?
 *    - `digit = num % 10; rev = (rev * 10) + digit; num /= 10;`
 *
 * 4. ALGORITHM EXPLANATION:
 * *   Step 1: Read integer `num` and save copy in `n`.
 *   Step 2: Extract last digit using `% 10` and build `rev = rev * 10 + digit`.
 *   Step 3: Strip last digit using `/ 10`.
 *   Step 4: If `n == rev`, number is Palindrome; else Not.
 * =====
 */

#include <iostream>

// Use standard library namespace globally
using namespace std;

// --- Main Execution Entry Point ---
int main() {
    int n, num, digit, rev = 0;

    // Output formatted data to console stream
    cout << "Enter a positive number: ";
    // Read user input from standard input stream
    cin >> num;

    n = num;

    // Loop execution block
    while (num > 0) {
        digit = num % 10;
        rev = (rev * 10) + digit;
        num = num / 10;
    }

    // Evaluate conditional decision logic
    if (n == rev)
        // Output formatted data to console stream
        cout << n << " is a Palindrome Number." << endl;
    // Alternative conditional branch
    else
        // Output formatted data to console stream
        cout << n << " is NOT a Palindrome Number." << endl;
}

```

```
// Return zero code indicating successful program finish
return 0;
}
```

Problem 11: PatternPrint

Problem Description & Objective:

Implementation of PatternPrint

C++ Code Implementation:

```

/*
 * Question:
 * Create a program to print a pyramid pattern of stars.
 */

/*
 * =====
 * CONCEPT EXPLORATION & ALGORITHM BREAKDOWN
 * =====
 * 1. WHICH CONCEPT IS USED?
 *    - Multi-loop Geometry (Spaces + Stars per row).
 *
 * 2. WHY IT IS USED?
 *    - Pyramid patterns require printing (rows - i) spaces followed by (2*i - 1) stars per row.
 *
 * 3. HOW IT IS USED?
 *    - Nested loops for spaces `j = 1..(rows-i)` and stars `k = 1..(2*i-1)`.
 *
 * 4. ALGORITHM EXPLANATION:
 * *   Step 1: Input total `rows`.
 *   Step 2: Outer loop `i` from 1 to `rows`.
 *   Step 3: Inner loop 1 prints spaces: `rows - i` times.
 *   Step 4: Inner loop 2 prints stars: `2*i - 1` times.
 *   Step 5: Print newline.
 * =====
 */

#include <iostream>

// Use standard library namespace globally
using namespace std;

// --- Main Execution Entry Point ---
int main() {
    int rows;
    // Output formatted data to console stream
    cout << "Enter number of rows: ";
    // Read user input from standard input stream
    cin >> rows;

    // Loop execution block
    for (int i = 1; i <= rows; i++) {
        // Loop execution block
        for (int j = 1; j <= rows - i; j++) {
            // Output formatted data to console stream
            cout << " ";
        }
        // Loop execution block
        for (int k = 1; k <= (2 * i - 1); k++) {
            // Output formatted data to console stream
            cout << "*";
        }
        // Output formatted data to console stream
        cout << endl;
    }

    // Return zero code indicating successful program finish

```

```
    return 0;  
}
```

Problem 12: PrimeNumber

Problem Description & Objective:

Implementation of PrimeNumber

C++ Code Implementation:

```

/*
 * Question:
 * Program to check whether a number is prime using while loop.
 */

/*
 * =====
 * CONCEPT EXPLORATION & ALGORITHM BREAKDOWN
 * =====
 * 1. WHICH CONCEPT IS USED?
 *    - Trial Division Primality Test up to Square Root (`i * i <= n`).
 *
 * 2. WHY IT IS USED?
 *    - Testing factors up to sqrt(N) optimizes primality testing from O(N) to O(sqrt(N)) time complexity.
 *
 * 3. HOW IT IS USED?
 *    - `while (i * i <= n)` checks if `n % i == 0`.
 *
 * 4. ALGORITHM EXPLANATION:
 * *   Step 1: Input integer `n`.
 *   Step 2: If `n <= 1`, set `isPrime = false`.
 *   Step 3: Loop `i = 2` while `i * i <= n`: if `n % i == 0`, set `isPrime = false` and `break`.
 *   Step 4: Print prime status.
 * =====
 */

#include <iostream>

// Use standard library namespace globally
using namespace std;

// --- Main Execution Entry Point ---
int main() {
    int n, i = 2;
    bool isPrime = true;

    // Output formatted data to console stream
    cout << "Enter a positive integer: ";
    // Read user input from standard input stream
    cin >> n;

    // Evaluate conditional decision logic
    if (n <= 1) {
        isPrime = false;
    } // Alternative conditional branch
    else {
        // Loop execution block
        while (i * i <= n) {
            // Evaluate conditional decision logic
            if (n % i == 0) {
                isPrime = false;
                break;
            }
            i++;
        }
    }
}

```

```
// Evaluate conditional decision logic
if (isPrime)
    // Output formatted data to console stream
    cout << n << " is a prime number." << endl;
// Alternative conditional branch
else
    // Output formatted data to console stream
    cout << n << " is NOT a prime number." << endl;

// Return zero code indicating successful program finish
return 0;
}
```

Problem 13: ReverseNumber

Problem Description & Objective:

Implementation of ReverseNumber

C++ Code Implementation:

```

/*
 * Question:
 * Program to reverse the digits of a number.
 */

/*
 * =====
 * CONCEPT EXPLORATION & ALGORITHM BREAKDOWN
 * =====
 * 1. WHICH CONCEPT IS USED?
 *    - Base-10 Shift & Extraction.
 *
 * 2. WHY IT IS USED?
 *    - Reverses integer representation using modulo and multiplication.
 *
 * 3. HOW IT IS USED?
 *    - `reversedNumber = reversedNumber * 10 + remainder;`
 *
 * 4. ALGORITHM EXPLANATION:
 *    * Step 1: Read `n` and store original.
 *    * Step 2: While `n != 0`: extract `remainder = n % 10`, update `reversedNumber = reversedNumber * 10
+ remainder`, reduce `n /= 10`.
 *    * Step 3: Output reversed number.
 * =====
 */

#include <iostream>

// Use standard library namespace globally
using namespace std;

// --- Main Execution Entry Point ---
int main() {
    int n, reversedNumber = 0, remainder;

    // Output formatted data to console stream
    cout << "Enter an integer: ";
    // Read user input from standard input stream
    cin >> n;

    int original = n;
    // Loop execution block
    while (n != 0) {
        remainder = n % 10;
        reversedNumber = reversedNumber * 10 + remainder;
        n /= 10;
    }

    // Output formatted data to console stream
    cout << "Reversed Number of " << original << " is: " << reversedNumber << endl;

    // Return zero code indicating successful program finish
    return 0;
}

```

Problem 14: SumOfDigit

Problem Description & Objective:

Implementation of SumOfDigit

C++ Code Implementation:

```

/*
 * Question:
 * Create a program that computes the sum of the digits of an integer.
 */

/*
 * =====
 * CONCEPT EXPLORATION & ALGORITHM BREAKDOWN
 * =====
 * 1. WHICH CONCEPT IS USED?
 *    - Digit Stripping Accumulator.
 *
 * 2. WHY IT IS USED?
 *    - Iteratively extracts individual digits and adds them to a running sum.
 *
 * 3. HOW IT IS USED?
 *    - `sum += temp % 10; temp /= 10;` inside a `while (temp > 0)` loop.
 *
 * 4. ALGORITHM EXPLANATION:
 * *   Step 1: Input `n` and take absolute value `abs(n)`.
 * *   Step 2: Loop while `temp > 0`: extract `m = temp % 10`, add to `sum`, integer divide `temp /= 10`.
 * *   Step 3: Output `sum`.
 * =====
 */

#include <iostream>

// Use standard library namespace globally
using namespace std;

// --- Main Execution Entry Point ---
int main() {
    int n, sum = 0, m;

    // Output formatted data to console stream
    cout << "Enter a number: ";
    // Read user input from standard input stream
    cin >> n;

    int temp = abs(n);
    // Loop execution block
    while (temp > 0) {
        m = temp % 10;
        sum += m;
        temp /= 10;
    }

    // Output formatted data to console stream
    cout << "Sum of digits: " << sum << endl;

    // Return zero code indicating successful program finish
    return 0;
}

```

Problem 15: SumPositive

Problem Description & Objective:

Implementation of SumPositive

C++ Code Implementation:

```

/*
 * Question:
 * Program using continue to sum all positive numbers entered by user, skipping negative ones.
 */

/*
 * =====
 * CONCEPT EXPLORATION & ALGORITHM BREAKDOWN
 * =====
 * 1. WHICH CONCEPT IS USED?
 *    - Conditional Accumulation with `continue`.
 *
 * 2. WHY IT IS USED?
 *    - Bypasses negative values cleanly during summation.
 *
 * 3. HOW IT IS USED?
 *    - `if (num < 0) continue; sum += num;`
 *
 * 4. ALGORITHM EXPLANATION:
 *    Step 1: Loop 5 times to read inputs.
 *    Step 2: Read `num`. If `num < 0`, skip iteration using `continue`.
 *    Step 3: Else add `num` to `sum`.
 *    Step 4: Output positive total sum.
 * =====
 */

#include <iostream>

// Use standard library namespace globally
using namespace std;

// --- Main Execution Entry Point ---
int main() {
    int sum = 0;

    // Output formatted data to console stream
    cout << "Enter 5 numbers:" << endl;
    // Loop execution block
    for (int i = 1; i <= 5; i++) {
        int num;
        // Read user input from standard input stream
        cin >> num;
        // Evaluate conditional decision logic
        if (num < 0) {
            // Output formatted data to console stream
            cout << "Skipping negative number: " << num << endl;
            continue;
        }
        sum += num;
    }

    // Output formatted data to console stream
    cout << "Sum of positive numbers: " << sum << endl;

    // Return zero code indicating successful program finish

```

```
    return 0;  
}
```

Problem 16: even numbers continue

Problem Description & Objective:

Implementation of even numbers continue

C++ Code Implementation:

```

/*
 * Question:
 * Create a program using continue to print only even numbers using continue for odd numbers.
 */

/*
 * =====
 * CONCEPT EXPLORATION & ALGORITHM BREAKDOWN
 * =====
 * 1. WHICH CONCEPT IS USED?
 *    - Filtering with Loop Control `continue`.
 *
 * 2. WHY IT IS USED?
 *    - Demonstrates skipping odd numbers in range 1..20.
 *
 * 3. HOW IT IS USED?
 *    - `if (i % 2 != 0) continue;`
 *
 * 4. ALGORITHM EXPLANATION:
 *    * Step 1: Loop `i` from 1 to 20.
 *    * Step 2: If `i` is odd, `continue`.
 *    * Step 3: Print even `i`.
 * =====
 */

#include <iostream>

// Use standard library namespace globally
using namespace std;

// --- Main Execution Entry Point ---
int main() {
    // Output formatted data to console stream
    cout << "Even numbers from 1 to 20:" << endl;
    // Loop execution block
    for (int i = 1; i <= 20; i++) {
        // Evaluate conditional decision logic
        if (i % 2 != 0) {
            continue;
        }
        // Output formatted data to console stream
        cout << i << " ";
    }
    // Output formatted data to console stream
    cout << endl;

    // Return zero code indicating successful program finish
    return 0;
}

```

Problem 17: multiplication table

Problem Description & Objective:

Implementation of multiplication table

C++ Code Implementation:

```
/*
 * Question:
 * Create a program using for loop multiplication table for a number.
 */

/*
 * =====
 * CONCEPT EXPLORATION & ALGORITHM BREAKDOWN
 * =====
 * 1. WHICH CONCEPT IS USED?
 *    - Formatted Loop Output.
 *
 * 2. WHY IT IS USED?
 *    - Prints table of n from 1 to 10.
 *
 * 3. HOW IT IS USED?
 *    - `for (int i = 1; i <= 10; i++) cout << n << 'x' << i << '=' << n*i;`
 *
 * 4. ALGORITHM EXPLANATION:
 * *   Step 1: Read `n`.
 *   Step 2: Loop `i` = 1..10.
 *   Step 3: Print `n * i`.
 * =====
 */

#include <iostream>

// Use standard library namespace globally
using namespace std;

// --- Main Execution Entry Point ---
int main() {
    int n;
    // Output formatted data to console stream
    cout << "Enter number: ";
    // Read user input from standard input stream
    cin >> n;

    // Loop execution block
    for (int i = 1; i <= 10; i++) {
        // Output formatted data to console stream
        cout << n << " x " << i << " = " << (n * i) << endl;
    }

    // Return zero code indicating successful program finish
    return 0;
}
```

Problem 18: positive number

Problem Description & Objective:

Implementation of positive number

C++ Code Implementation:

```
/*
 * Question:
 * Create a program that prompts the user to enter a positive number using do-while loop.
 */

/*
 * =====
 * CONCEPT EXPLORATION & ALGORITHM BREAKDOWN
 * =====
 * 1. WHICH CONCEPT IS USED?
 *    - Input Validation Post-Test Loop (`do-while`).
 *
 * 2. WHY IT IS USED?
 *    - `do-while` guarantees execution at least once before checking if input is valid.
 *
 * 3. HOW IT IS USED?
 *    - `do { cin >> number; } while (number <= 0);`
 *
 * 4. ALGORITHM EXPLANATION:
 * *   Step 1: Execute `do` block: prompt user and read `number`.
 *   Step 2: Evaluate `while (number <= 0)`.
 *   Step 3: Repeat if number is not positive.
 *   Step 4: Output confirmed positive number.
 * =====
 */

#include <iostream>

// Use standard library namespace globally
using namespace std;

// --- Main Execution Entry Point ---
int main() {
    int number;

    do {
        // Output formatted data to console stream
        cout << "Please enter a positive number: ";
        // Read user input from standard input stream
        cin >> number;
    } while (number <= 0);

    // Output formatted data to console stream
    cout << "Thank you! You entered a valid positive number: " << number << endl;

    // Return zero code indicating successful program finish
    return 0;
}
```

Problem 19: prime number

Problem Description & Objective:

Implementation of prime number

C++ Code Implementation:

```

/*
 * Question:
 * Create a program using for loop to display if a number is prime or not.
 */

/*
 * =====
 * CONCEPT EXPLORATION & ALGORITHM BREAKDOWN
 * =====
 * 1. WHICH CONCEPT IS USED?
 *   - For-Loop Primality Test (`i * i <= n`).
 *
 * 2. WHY IT IS USED?
 *   - Fast factor check up to square root of n using a `for` loop.
 *
 * 3. HOW IT IS USED?
 *   - `for (int i = 2; i * i <= n; i++) if (n % i == 0) { isPrime = false; break; }`
 *
 * 4. ALGORITHM EXPLANATION:
 * *   Step 1: Read `n`.
 *   Step 2: Check `n <= 1`.
 *   Step 3: Loop `i` from 2 up to `i * i <= n`. If divisible, set `isPrime = false` and `break`.
 *   Step 4: Display Prime or Not Prime.
 * =====
 */

#include <iostream>

// Use standard library namespace globally
using namespace std;

// --- Main Execution Entry Point ---
int main() {
    int n;
    bool isPrime = true;

    // Output formatted data to console stream
    cout << "Enter integer: ";
    // Read user input from standard input stream
    cin >> n;

    // Evaluate conditional decision logic
    if (n <= 1) {
        isPrime = false;
    } // Alternative conditional branch
    else {
        // Loop execution block
        for (int i = 2; i * i <= n; i++) {
            // Evaluate conditional decision logic
            if (n % i == 0) {
                isPrime = false;
                break;
            }
        }
    }

    // Evaluate conditional decision logic

```

```
if (isPrime) {  
    // Output formatted data to console stream  
    cout << n << " is Prime" << endl;  
    // Alternative conditional branch  
} else {  
    // Output formatted data to console stream  
    cout << n << " is Not Prime" << endl;  
}  
  
// Return zero code indicating successful program finish  
return 0;  
}
```

Problem 20: square break loop

Problem Description & Objective:

Implementation of square break loop

C++ Code Implementation:

```

/*
 * Question:
 * Write a program that continuously reads integers and prints their squares until -1 is entered.
 */

/*
 * =====
 * CONCEPT EXPLORATION & ALGORITHM BREAKDOWN
 * =====
 * 1. WHICH CONCEPT IS USED?
 *    - Infinite Loop Exit Sentinel (`while(true)` + `break`).
 *
 * 2. WHY IT IS USED?
 *    - Continuous processing loop until user enters exit sentinel `-1`.
 *
 * 3. HOW IT IS USED?
 *    - `if (val == -1) break;` inside `while(true)`.
 *
 * 4. ALGORITHM EXPLANATION:
 *    * Step 1: Start `while(true)` loop.
 *    * Step 2: Read `val`.
 *    * Step 3: If `val == -1`, break out of loop.
 *    * Step 4: Output `val * val`.
 * =====
 */

#include <iostream>

// Use standard library namespace globally
using namespace std;

// --- Main Execution Entry Point ---
int main() {
    int val;
    // Output formatted data to console stream
    cout << "Square Calculator (Enter -1 to quit)" << endl;

    // Loop execution block
    while (true) {
        // Output formatted data to console stream
        cout << "Enter a number: ";
        // Read user input from standard input stream
        cin >> val;

        // Evaluate conditional decision logic
        if (val == -1) {
            // Output formatted data to console stream
            cout << "Exiting program..." << endl;
            break;
        }

        // Output formatted data to console stream
        cout << "Square of " << val << " is: " << (val * val) << endl;
    }

    // Return zero code indicating successful program finish

```

```
    return 0;  
}
```

Problem 21: sum positive continue

Problem Description & Objective:

Implementation of sum positive continue

C++ Code Implementation:

```

/*
 * Question:
 * Create a program using continue to sum all positive numbers entered by the user; skip any negative
numbers.
 */

/*
 * =====
 * CONCEPT EXPLORATION & ALGORITHM BREAKDOWN
 * =====
 * 1. WHICH CONCEPT IS USED?
 *   - Loop Skip Control Flow (`continue`).
 *
 * 2. WHY IT IS USED?
 *   - Ignores invalid negative numbers in a dynamic count loop.
 *
 * 3. HOW IT IS USED?
 *   - `if (num < 0) continue; totalSum += num;`
 *
 * 4. ALGORITHM EXPLANATION:
 * *   Step 1: Input entry count.
 *   Step 2: Loop count times: read `num`. If `num < 0`, `continue`.
 *   Step 3: Add positive `num` to `totalSum`.
 *   Step 4: Print `totalSum`.
 * =====
 */

#include <iostream>

// Use standard library namespace globally
using namespace std;

// --- Main Execution Entry Point ---
int main() {
    int count, num, totalSum = 0;

    // Output formatted data to console stream
    cout << "How many numbers would you like to enter? ";
    // Read user input from standard input stream
    cin >> count;

    // Loop execution block
    for (int i = 1; i <= count; i++) {
        // Output formatted data to console stream
        cout << "Enter number " << i << ": ";
        // Read user input from standard input stream
        cin >> num;

        // Evaluate conditional decision logic
        if (num < 0) {
            // Output formatted data to console stream
            cout << "Negative number skipped." << endl;
            continue;
        }

        totalSum += num;
    }
}

```

```
// Output formatted data to console stream
cout << "Total sum of positive numbers: " << totalSum << endl;

// Return zero code indicating successful program finish
return 0;
}
```

Problem 22: sum until zero

Problem Description & Objective:

Implementation of sum until zero

C++ Code Implementation:

```

/*
 * Question:
 * Develop a program that calculates the sum of all numbers entered by a user until the user enters 0.
 */

/*
 * =====
 * CONCEPT EXPLORATION & ALGORITHM BREAKDOWN
 * =====
 * 1. WHICH CONCEPT IS USED?
 *    - Post-Condition Accumulation Loop (`do-while`).
 *
 * 2. WHY IT IS USED?
 *    - Accumulates numbers continuously until sentinel `0` is typed.
 *
 * 3. HOW IT IS USED?
 *    - `do { cin >> number; sum += number; } while (number != 0.0);`
 *
 * 4. ALGORITHM EXPLANATION:
 * *   Step 1: Enter `do` loop.
 *   Step 2: Read `number` and add to `sum`.
 *   Step 3: Repeat while `number != 0.0`.
 *   Step 4: Display total `sum`.
 * =====
 */

#include <iostream>

// Use standard library namespace globally
using namespace std;

// --- Main Execution Entry Point ---
int main() {
    double number, sum = 0.0;

    // Output formatted data to console stream
    cout << "Enter numbers to add (enter 0 to finish):" << endl;

    do {
        // Read user input from standard input stream
        cin >> number;
        sum += number;
    } while (number != 0.0);

    // Output formatted data to console stream
    cout << "Total sum: " << sum << endl;

    // Return zero code indicating successful program finish
    return 0;
}

```

6. Exam Scoring Points & Memory Revision

★ Must-Write Points

1. Loop bounds and off-by-one errors: Always verify boundary conditions ($i < N$ vs $i \leq N$).
2. Fibonacci, Prime checks, and GCD/LCM utilize loop state accumulation.
3. Range-based for loop: `for (const auto& elem : container)` provides safe iteration.
4. Time complexity of single loops is $O(N)$; nested double loops evaluate to $O(N^2)$.
5. Loop invariant maintenance ensures correctness of iterative algorithms.

⚡ Quick Recall

Entry Test (for/while) -> Iteration Block -> Exit Test (do-while) -> Range Loop -> $O(N)$ Bounds